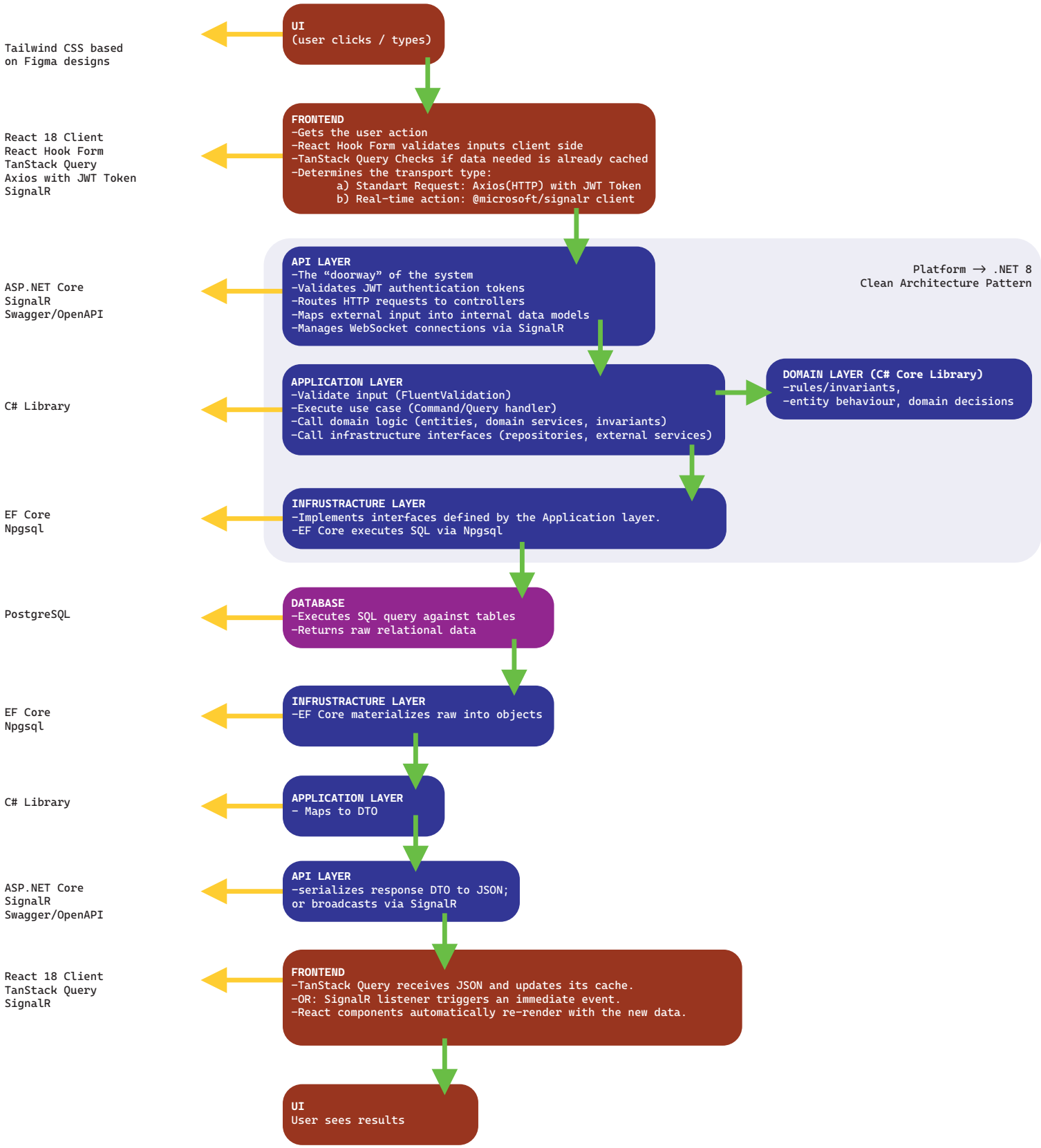
The SOCIAL
flow & technical tools



Note: HTTP is used for CRUD endpoints; SignalR is used for chat, notifications, and online status (presence).

The SOCIAL
social network – work seperation

Note: Longer colors mean the owner of the module
shorter color mean the supporter of the module

Person suggestions:
Person 1 → Daria
Person 2 → Deniz
Person 3 → Valeriy
Person 4 → Micha

PERSON 1: BACKEND FOUNDATION (INFRASTRUCTURE + DB + “IT RUNS”)

Tasks:
- Postgres setup (Docker Compose or documented local setup)
- EF Core integration + migrations (if you are taking the ORM minor)
- Repository implementations in Infrastructure
- CI build check (at least dotnet build on Ubuntu)
- This person owns “persistence is reliable”.

PERSON 2: BACKEND FEATURES (APPLICATION + DOMAIN) FOR “SOCIAL CORE”

Tasks:
- Profile endpoints (view/update)
- Friends endpoints (add/remove/list)
- Domain rules and invariants (for example: cannot friend yourself; cannot duplicate friendships)
- DTO mapping and validation (FluentValidation can live here)
- This person owns “Major: users interact with users” except realtime chat.

PERSON 3: REALTIME + NOTIFICATIONS (SIGNALR, PRESENCE)

Tasks:
- SignalR hub(s) for chat and presence
- Chat send/receive flow (store message, broadcast, acknowledge)
- Notification events and storage rules (minor notification system)
- Connection/disconnection handling, online status
- This person owns “Major: realtime features” and notification plumbing.

PERSON 4: FRONTEND (REACT)

Tasks:
- React app skeleton: routing, auth state, API client, error handling
- Pages: login, profile, friends list, chat UI
- i18n if you pick that minor (at least 3 languages)
- Integrate REST + SignalR client, with a stable pattern (TanStack Query for REST, SignalR listener for pushes)

The SOCIAL
social network – modules

2. MAJOR
Owner: Person 3 (Realtime + notifications).
Primary responsibilities (Person 3):
-Implement SignalR/WebSocket hubs (chat, presence, notification push).
-Connection lifecycle: connect, disconnect, reconnect, heartbeat (if needed).
-Broadcasting strategy: groups, targeted users, efficient fan-out.

Secondary contributors:
-**Person 1** (Backend foundation): supports DB persistence for chat/messages, any Redis/backplane decision (if used), Docker/hosting config.
-**Person 4** (Frontend): implements SignalR client, subscribes to events, updates UI state.
-**Person 2** (Backend features): defines use-cases and domain rules for chat/notification actions (for example “can these users message each other?”), if those rules exist.

3. MAJOR (SEPERATED BY FEATURE)
PROFILE SYSTEM (VIEW USER INFORMATION)
Owner: Person 2 (Backend features)
Domain model (User/Profile), use-cases, DTOs, validation, permissions.
Support: Person 4 (Frontend)
Profile page, fetch/update UI, caching, i18n strings.

FRIEND SYSTEM (ADD/REMOVE/LIST)
Owner: Person 2 (Backend features)
Friendship model, rules (no duplicates, no self-friend), endpoints, DTOs.
Support: Person 4 (Frontend)
Friends list UI, add/remove actions, state updates.

BASIC CHAT SYSTEM (SEND/RECEIVE)
This is mainly two parts:
Realtime transport + presence
Owner: Person 3 (Realtime)
SignalR hub, connection/disconnection, broadcasting, groups.
Chat domain + persistence + API endpoints
Owner: Person 2 (Backend features) for use-cases and rules (who can message whom, validation).
Support: Person 1 (Backend foundation) for EF Core schema/migrations and repository implementation.
Chat UI
Owner: Person 4 (Frontend) for chat screens and SignalR client integration.

4. MAJOR
Authentication (login/session/JWT, protected endpoints)
Owner: Person 1 (Backend foundation): Configure ASP.NET Core auth, token/cookie strategy, password hashing, user identity storage, middleware.
Support: Person 4 (Frontend): Login UI, token/cookie handling, auth state, route guarding.

Update profile information + profile page
Backend owner: Person 2 (Backend features): Use-cases/endpoints: get profile, update profile, validation, DTOs, permissions.
Frontend owner: Person 4 (Frontend): Profile page UI and edit form.

Upload avatar + default avatar
Owner: Person 1 (Backend foundation) for storage and security: File upload endpoint, validation (type/size), storage location, access control, default avatar handling.
Support: Person 2 (Backend features): Profile model fields and rules (avatar URL/id), update profile workflow.
Frontend owner: Person 4: Upload UI, preview, progress indicator, fallback to default.

Friends + online status
Friends CRUD
Owner: Person 2 (Backend features) (same as earlier)
Support: Person 4 (UI)
Online status
Owner: Person 3 (Realtime) (presence tracking via SignalR)
Support: Person 2 (expose “is online” in profile/friends response if needed)
Support: Person 4 (display status in UI)

Note: We will have to work together a lot and interchangeably

The SOCIAL
social network - modules

1. MINOR
Owner: Person 3 (core owner).
Supporter: Person 2(bussiness rules and triggering), Person1(persistence), Person 4(frontend UX)

2. MINOR
Owner: Person 1 (Backend foundation)
Implement upload endpoints, server-side validation (type/size), secure storage (local or S3-like), access control, delete, and metadata persistence (EF Core tables).
Support: Person 4 (UX)
Upload UI (picker, preview), progress indicator, client-side validation, and “manage files” screens (list/download/delete).
Support: Person 2
Wire uploads into use-cases (avatar, post images, document attachments) and enforce domain rules about who can attach/see/delete.

4. MINOR
Owner: Person 1 (Backend foundation)
Configure ASP.NET Core authentication for OAuth providers, handle callback flow, create/link local users, store external login mapping, manage secrets/config.
Support: Person 4 (UX)
“Sign in with Google/GitHub/42” UI, redirect flow, handle post-login state.
Support: Person 2
Domain/Application rules for linking accounts (email already exists, merge policy, first-time user profile creation).

6. MINOR
Owner: Person 4 (Frontend)

7. MINOR
Owner: Person 1 (Backend foundation)
et up EF Core with PostgreSQL (Npgsql), create DbContext, entity mappings/configurations, migrations, connection strings/environment config, and ensure the API can run against the DB locally and in the school environment.
Support: Person 2
align domain needs with persistence (which entities/relations you actually need), help define queries (includes, pagination), and shape repository interfaces so use-cases can read/write via EF Core.

Note: We will have to work together a lot and interchangeably

The SOCIAL
social network - modules

MAJOR

1. Major: Use a framework for both the frontend and backend.
- Use a frontend framework (React, Vue, Angular, Svelte, etc.).
- Use a backend framework (Express, NestJS, Django, Flask, Ruby on Rails, etc.).
- Full-stack frameworks (Next.js, Nuxt.js, SvelteKit) count as both if you use both their frontend and backend capabilities.

2. Major: Implement real-time features using WebSockets or similar technology.
- Real-time updates across clients.
- Handle connection/disconnection gracefully.
- Efficient message broadcasting.

3. Major: Allow users to interact with other users. The minimum requirements are:
- A basic chat system (send/receive messages between users).
- A profile system (view user information).
- A friend's system (add/remove friends, see friends list).

4. Major: Standard user management and authentication.
- Users can update their profile information.
- Users can upload an avatar (with a default avatar if none is provided).
- Users can add other users as friends and see their online status.
- Users have a profile page displaying their information.

MINOR

1. • Minor: A complete notification system for all creation, update, and deletion actions.

2. Minor: File upload and management system.
- Support multiple file types (images, documents, etc.).
- Client-side and server-side validation (type, size, format).
- Secure file storage with proper access control.
- File preview functionality where applicable.
- Progress indicators for uploads.
- Ability to delete uploaded files.

3. Minor: Support for multiple languages (at least 3 languages).
- Implementan i18n (internationalization) system.
- At least 3 complete language translations.
- Language switcher in the UI.
- All user-facing text must be translatable.

4. Minor: Implement remote authentication with OAuth 2.0 (Google, GitHub, 42, etc.).

5. Minor: Implement a complete 2FA (Two-Factor Authentication) system for the users.

6. Minor: Custom-made design system with reusable components, including a proper color palette, typography, and icons (minimum: 10 reusable components)

7. Minor: Use an ORM for the database

Note: We will have to work together a lot and interchangeably